

Integrative Programming 2 MIDTERMS

TOPIC 5 – WEB APPLICATIONS & VUE.JS

WEB APPLICATION

- A web application is a **dynamic**, interactive software that runs in a web browser.
- Unlike static websites, web apps allow users to perform tasks such as submitting forms and real-time updates.

Examples:

Gmail – Email client
Trello – Task management
Netflix – Streaming platform

KEY DIFFERENCES FROM TRADITIONAL WEBSITES

1. **Web Applications** – **Dynamic** content that changes based on user input; requires backend APIs such as REST and GraphQL; provides rich interactivity similar to desktop applications.
2. **Traditional Websites** – **Static** content with fixed HTML pages; mostly server-rendered HTML; limited interactivity such as links and basic forms.

WHY USE FRONTEND FRAMEWORKS?

Problems with Vanilla JavaScript:

- Complex DOM manipulation – Hard to maintain.
- No structured architecture – Can result in spaghetti code.
- Poor reusability – May require copy-pasting components.

BENEFITS OF VUE.JS

1. **Reactivity** – UI updates automatically when data changes.
2. **Component-based architecture** – Provides reusable UI blocks.
3. **Lightweight & flexible** – Can be incrementally adopted.

SPA ADVANTAGES

- Faster navigation – No full-page reloads.
- Better UX – Feels like a native application.

CLIENT-SIDE VS SERVER-SIDE

Client-Side Rendering (SPA):

- Rendered in the browser using JavaScript.
- Initial load fetches a single HTML + JS bundle.

Advantage:

- Smoother UX and offline capabilities.

Disadvantage:

- Slower initial load and SEO challenges.

Server-Side Rendering (MPA):

- Rendered on the server and returns full HTML.
- Each navigation requests a new HTML page. .

Advantage:

- Better SEO and faster initial load.

Disadvantage:

- Full page reloads and less interactive.

CORE CONCEPTS OF VUE.JS

1. Reactive Data Binding

- Changes to data automatically update the UI.

2. Directives

- Vue directives are special attributes used to apply dynamic behavior to HTML elements.

v-if – Conditional rendering.

v-for – Looping through lists.

v-model – Two-way binding for forms.

v-bind – Dynamic attributes.

v-else – Executes if the v-if condition is not satisfied.

TOPIC 6 – JAVASCRIPT EVENTS

EVENTS

- Events in JavaScript are actions or occurrences that happen in the web browser.
- These actions can be triggered by user actions, browser actions, or programmatic triggers.

1. **User Actions** – Clicking a button, typing on the keyboard, moving the mouse, etc.

2. **Browser Actions** – Page loading or unloading, resizing the browser window, etc.

3. **Programmatic Triggers** – Events triggered by JavaScript code itself, such as simulating a click.

EVENT TYPES

- **Mouse Events** – Events that occur when the user interacts with the mouse or a pointing device.
- **Keyboard Events** – Events that occur when the user interacts with the keyboard.
- **Click Events** – Specific to clicks and are part of mouse events. They are triggered when a user clicks on an element such as a button or link.
- **Change Events** – Triggered when the value of an input, select, or textarea element changes. They help detect when the user changes the value in a form element.

MOUSE EVENTS

- **click** – Triggered when the user clicks an element.
- **dblclick** – Triggered when the user double-clicks.
- **mouseover** – Triggered when the mouse pointer moves over an element.
- **mouseout** – Triggered when the mouse pointer leaves an element.

KEYBOARD EVENTS

- Keyboard events occur when the user interacts with the keyboard.
- **keypress** – Fired when the user presses a key, except for special keys like Shift or Ctrl.
- **keydown** – Fired when any key is pressed down.
- **keyup** – Fired when the user releases a key.

CHANGE EVENTS

- Change events are triggered when the value of an input, select, or textarea element changes.
- They are useful for detecting when the user changes the value of a form element.

ADDING AND REMOVING EVENTS

- **addEventListener()** – Used to add an event listener to an element.
- **removeEventListener()** – Used to remove an event listener from an element.
- **Event Listener** – A function or mechanism that waits for a specific event to occur and then executes the associated code.

Important: The same function reference used when adding an event listener must be used when removing it.

TOPIC 6 – SINGLE PAGE APPLICATIONS VIA VUEJS

SINGLE PAGE APPLICATION (SPA)

- A single HTML page where content is dynamically updated via JavaScript.

HOW SPA WORKS

- Initial load → Fetches index.html + JS bundle.
- Subsequent navigation → Only fetches data; there is no full page reload.

VUE ROUTER ROLE

- Vue Router manages different views such as /home and /about within the same page.

VUE ROUTER SETUP

- Vue Router is used to implement client-side routing in a Vue.js Single Page Application.

ROUTER CONFIGURATION

- createRouter() – Creates and configures the Vue Router.
- createWebHashHistory() – Enables client-side routing using URL hashes (#).
- routes – Maps URL paths to components.

Example:

```
{ path: '/', component: Home }  
{ path: '/about', component: About }
```

ROUTE

- A route is a URL path mapped to a specific Vue component.

Example:

```
"/" → Home  
"/about" → About  
"/contact" → Contact
```

router-link

- Used to create navigation links between routes.

Example:

```
<router-link  
to="/">Home</router-link>  
<router-link  
to="/about">About</router-link>  
<router-link  
to="/contact">Contact</router-link>
```

router-view

- Used as the location where the dynamic content of the current route is displayed.

Example:

```
<router-view></router-view>
```

COMPONENTS

- Components are reusable parts of a Vue application.

Example:

```
Home – Home Page Content  
About – About Page Content  
Contact – Contact Page Content
```

ROUTER INITIALIZATION

Create the routes.

Create the router using createRouter().

Configure createWebHashHistory().

Use the router with app.use(router).

Mount the application.

Example:

```
const app = Vue.createApp({});  
app.use(router);  
app.mount('#app');
```

COMPOSITION API IN VUE ROUTER

- The Composition API is a modern syntax in Vue 3 for organizing component logic.

Benefits:

- Better code organization.
- Reusable logic.

SPA PROS

- Faster navigation because there are no full-page reloads.
- Better UX through fluid transitions.

SPA CONS

- SEO challenges.
- Initial load delay.
- SEO challenges can be addressed using SSR.
- Initial load can be optimized using code-splitting.

TOPIC 8 – IMPLEMENTING PASSWORD HASHING TECHNIQUES IN PHP & MYSQL WITH VUE 3

INTRODUCTION

- Authentication is the first line of defense in web systems.
- Plain-text password storage creates severe vulnerabilities.
- Database leaks can expose credentials across multiple platforms due to password reuse.
- Password hashing secures stored credentials to mitigate breaches.

WHAT IS PASSWORD HASHING?

- Password hashing is a **one-way mathematical transformation** producing a fixed-length digest.
- Unlike encryption, hashing is designed to be irreversible.
- Stored hashes protect user credentials from being viewed by anyone.

Example:

"mypassword" → 2y10\$J3ax...
(bcrypt/Argon2 style)

WHY NOT STORE PLAIN TEXT?

- Anyone with database access can read actual passwords.
- Leaked databases lead to immediate credential compromise.
- Users often reuse passwords, which can lead to cross-site account takeovers.
- Regulatory frameworks such as GDPR and PCI DSS require secure storage.

HASHING VS ENCRYPTION

- **Encryption** – Two-way process; reversible with a key; ideal for data at rest or in transit.
- **Hashing** – One-way function; irreversible; ideal for authentication.

- Authentication systems rely on hashing rather than reversible encryption

PHP HASHING FUNCTIONS

- `password_hash()` – Modern and secure; supports bcrypt/Argon2; automatically salts passwords to prevent rainbow tables.
- `PASSWORD_DEFAULT` – Selects a strong algorithm and cost automatically.
- `md5()` – Fast but cryptographically broken; should not be used for passwords.

`password_verify()`

- Hashes differ each time because of unique salts.
- Direct string comparison fails because a password can produce different hashes.
- `password_verify()` safely checks a plain input against a stored hash.

Returns boolean:

- **True** – Password matches the hash.
- **False** – Password does not match the hash

REGISTRATION FLOW

- User enters username and password in the frontend form.
- Backend PHP receives the data and applies `password_hash()`.
- Username and resulting hash are stored in MySQL.
- The plain password is never stored.
- If the database is compromised, attackers obtain hashes instead of raw passwords.

LOGIN FLOW

- User submits username and password.
- Backend fetches the stored hash for the user from MySQL.
- `password_verify()` compares the entered password with the stored hash.
- On success → Create a session.
- On failure → Return an authentication error.

VUE 3 WITH CDN

- Vue 3 can be loaded directly through a CDN.
- No Node.js or build tools are required.
- Useful for small projects and quick prototyping.
- **CDN** – Content Delivery Network.
- **v-model** – Used for two-way binding.
- **fetch/AJAX** – Used to communicate with PHP.

SECURITY BEST PRACTICES

- Use `password_hash()` with `bcrypt/Argon2` and `PASSWORD_DEFAULT`.
- Avoid MD5/SHA1 for passwords.
- Always use prepared statements such as PDO/MySQLi to prevent SQL injection.
- Use HTTPS to protect credentials in transit.
- Implement rate limiting/CAPTCHA to reduce brute-force attempts.
- Consider password rehashing with `password_needs_rehash()` when the algorithm or cost changes.

AUTHENTICATION WORKFLOW

- Vue (CDN) collects credentials through a form and sends them to PHP using `fetch/AJAX`.
- PHP validates input and interacts with MySQL using prepared statements.

- Registration: `password_hash()` → store hash.
- Login: fetch stored hash → `password_verify()` → session management.
- The backend can return JSON or text responses for Vue to display to the user.

ADVANTAGES OF THIS METHOD

- Strong resistance to rainbow table and brute-force attacks through adaptive hashing.
- Simple PHP APIs can reduce implementation errors.
- Vue 3 through CDN provides lightweight interactivity without a build step.
- The approach is compatible with best practices and compliance requirements.

CONCLUSION

- Password hashing is essential for secure user authentication.
- PHP provides `password_hash()` and `password_verify()` for practical and strong security.
- Vue 3 through CDN simplifies frontend integration for registration and login flows.
- Together, PHP, MySQL, password hashing, and Vue 3 form a modern, maintainable, and secure authentication foundation.